实验 2 Linux 网络编程基础

实验目的

1、认识结构体中地址对齐规则并能够运用到结构体的设计中。

2、掌握主机字节序和网络字节序的概念，能够完成主机字节序和网络字节序的互相转换。

3、掌握点分十进制形式 IP 地址与二进制形式 IP 地址的互相转换。

4、掌握通过 gethostbyname()获取主机信息的方法。

实验内容

1、结构体成员地址对齐及应用。

2、主机字节序与网络字节序互相转换。

3、点分十进制形式 IP 地址与二进制形式 IP 地址互相转换。

4、获取主机信息。

实验指导

任务一：结构体成员地址对齐及应用

1、原理

在对程序进行编译时，为了提高对结构体中成员变量的访问速度，使得对每一个成员变量的访问能够在一个存储器访问周期中完成，需要对结构体成员变量做地址对齐（alignment）处理。即结构体中每一个成员变量的起始偏移地址必须满足特定的规则，使得结构体成员变量的起始地址对齐到特定的边界。因而在存储结构体各成员变量时，可能会在成员变量之间加入一些空白填充字节（Padding），使得结构体的大小并不等于各成员变量大小之和，且各成员变量之间可能会出现填充字节而导致存储空间不连续。

以下程序清单 0-1 即是一个地址对齐的例子。程序中声明的三个结构体都各自拥有三个成员变量：一个 char 类型、一个 short int 类型和一个 int 类型。其中：char 数据类型的长度为一个字节，

short int 数据类型的长度为 2 字节，int 数据类型的长度为 4 字节。看起来这三个结构体的长度似乎都应该是 7 个字节。其实不然，通过程序的执行结果来看，d1 的长度实际为 8 个字节，d2 和 d3 的长度均为 12 个字节；通过观察还会发现，每个结构体中的三个成员变量在内存中并未存储在连续的地址空间，而是彼此之间存在间隙。

为了避免由于地址对齐原因而在各成员变量之间插入空白字节，可以采取如下方法：

(1) 通过合理安排各成员变量的类型、在结构体中的排列次序以及成员变量的个数，以避免在成员变量之间以及结构体的尾部插入空白字节。

(2) 使用预处理指令：#pragma pack(n)，要求按 n 个字节进行地址对齐。如果把 n 设置为 1，即要求按 1 个字节进行对齐处理，实际即为禁止地址对齐优化。

程序清单 0-1 地址对齐示例

```
#include <netinet/in.h>
int main()
{
    char *p1;
    short int *p2;
    int *p3;
    p1=&(d1.a);
    p2=&(d1.b);
    p3=&(d1.c);
    printf("%ld,%lx,%lx,%lx\n",sizeof(d1),(unsigned long int)(p1),(unsigned long int)(p2),(unsigned long int)(p3));

    p1=&(d2.a);
    p2=&(d2.b);
    p3=&(d2.c);
    printf("%ld,%lx,%lx,%lx\n",sizeof(d2),(unsigned long int)(p1),(unsigned long int)(p3),(unsigned long int)(p2));

    p1=&(d3.a);
    p2=&(d3.b);
    p3=&(d3.c);
    printf("%ld,%lx,%lx,%lx\n",sizeof(d3),(unsigned long int)(p2),(unsigned long int)(p3),(unsigned long int)(p1));
    return 0;
}

/*程序运行结果*/
8,55d3082ad018,55d3082ad01a,55d3082ad01c
12,55d3082ad030,55d3082ad034,55d3082ad038
12,55d3082ad020,55d3082ad024,55d3082ad028
```

2、实验内容及步骤

设有如下结构体：

```
struct {
   char ch_1;
   int int_32_1;
   unsigned short int int_16_1;
   unsigned int int_32_2;
   unsigned short int int_16_2;
   unsigned char ch2;
   short int int_16_3;
} sample;
```

(1) 请分析并给出该结构体在内存中的存储结构，使用程序对你的分析结果进行验证。

(2) 请总结地址对齐的规则。

(3) 若要避免在成员之间插入填充的空白字节，请问要如何对该结构体进行修改？

任务二：网络字节序和主机字节序的转换

1、原理

计算机内存一般会按字节进行编址，即每个字节对应一个地址。若一个数据由多个字节构成，那么在把该数据保存到计算机内存时，存在字节存储顺序的问题，即：是先保存数据中的高位字节还是先保存数据中的低位字节；多字节数据映像到内存时，其字节顺序随 CPU 的不同而有不同的组织方式（字节序）。这种与 CPU 相关的多字节在内存中的存储顺序称为主机字节序。

由于 double、float 等数据类型的存储结构有相应的国际标准，这些数据类型的存储不随 CPU 的不同而不同。因此，大端字节序和小端字节序主要是针对整型数据类型而言。

基本上存在两种类型的字节序：

(1) **大端字节序：**数据高有效位（字节）存放在内存的低地址，而数据低有效位（字节）存放于高内存地址的存储方式

(2) **小端字节序：**数据低有效字节存放在低内存地址，而数据高有效字节存放于高内存地址的存储方式。

所以在按内存地址从小到大的顺序依次访问内存单元时，对于大端字节序的主机来说是先访问到数据中的高位字节，再访问到数据中的低位字节；对于采用小端字节序的主机而言，则是先访问

到数据中的低位字节，再访问到数据的高位字节。

多字节数据在网络传输时，同样存在先传输数据中的高位字节，再传输数据中的低位字节；还是先传输数据的低位字节，再传输高位字节的问题。网络的协议标准规定是先传输数据高位字节，再传输数据的低位字节，该数据传输的字节顺序称作网络字节序。因此，网络字节序实际上就是大端字节序。

若主机字节序为大端字节序，其字节序与网络字节序一致，那么，网络通信程序在发送数据时，可以直接把缓冲区的内容直接传输到网络上；从网络中接收到的数据保存到缓冲区后，就可以直接使用，不需要做额外的处理。若主机字节序为小端字节序，那么，在发送数据之前要先把缓冲区中的多字节数据转换为网络字节序后才可以发送，接收到的数据也要先从网络字节序转换为主机字节序后才可以进行处理。

在编写网络通信程序时，并不能事先知道运行程序的目标平台采用的是大端字节序还是小端字节序，因而不能事先决定是否需要在主机字节序和网络字节序之间进行转换，只有在程序运行时，才能够确定目标平台的主机字节序、才能决定是否要进行主机字节序与网络字节序之间的转换。即：字节序的转换工作需要推迟到程序运行时来解决。

实际编写网络通信程序时，处理方案是：发送数据之前，先调用套接字接口函数把多字节数据转换成网络字节序再发送到网络上；从网络上接收到的数据先经过套接字接口函数转换为主机字节序后再进行处理。但是要不要真正进行字节序的转换，留待程序运行时，由被调用的套接字接口函数根据程序运行的硬件平台来决定。

套接字编程接口中与字节序转换相关的有以下函数：

```
#include <netinet/in.h>
/*32位无符号整型数由主机字节序转换为网络字节序*/
unsigned long htonl(unsigned long host_long);
/*16位无符号整型数由主机字节序转换为网络字节序*/
unsigned short htons(unsigned short host_short);
/*32位无符号整型数由网络字节序转换为主机字节序*/
unsigned long ntohl(unsigned long net_long);
/*16位无符号整型数由网络字节序转换为主机字节序*/
unsigned short ntohs(unsigned short net_short);
```

2、实验内容及步骤

(1) 通过编写程序来测试本机的主机字节序。

(2) Internet 域使用的套接字地址结构如下：

```
typedef uint32_t in_addr_t;
typedef uint16_t in_port_t;
struct in_addr
{
   in_addr_t  s_addr;
}
struct sockaddr_in
{
   sa_family_t  sin_family;
   in_port_t  sin_port;
   struct in_addr  sin_addr;

unsigned char sin_zero[sizeof (struct sockaddr)
 - (sizeof (unsigned short int))
 - sizeof (in_port_t)
 - sizeof (struct in_addr)];
};
```

其中地址族使用主机字节序，取值为 AF_INET；端口号使用网络字节序；IP 地址采用网络字节序。struct sockaddr_in 结构体的一般用法为：

- 定义一个 struct sockaddr_in 结构体变量。比如：

```
struct sockaddr_in addr;
```

- 将该结构体清零。比如：

```
memset(&addr,0,sizeof(struct sockaddr_in));
```

- 为该变量的各成员赋值。比如：

```
addr.sin_family = AF_INET;
addr.sin_port = htons(2000);          /*端口号为2000，需要转换为网络字节序后保存到结构体中*/
addr.sin_addr.s_addr = htonl(0xc0a80a64l);      /*IP地址为192.168.10.100。转换为网络字节序并保存*/
```

- 在调用套接字接口函数时，将该地址结构强制转换为 struct sockaddr 结构。比如：

```
int addrlen;
addrlen=sizeof(struct sockaddr_in)
bind(sock, (struct sockaddr)addr,&addrlen);
```

请定义一个 Internet 域的套接字地址结构体变量 my_addr，设置其端口号 5000、IP 地址为 0xC0A80101。

(3) 请解释为什么 struct sockaddr_in 结构体中各成员变量采用了这样的排列次序。

任务三：IP 地址的转换

1、原理

IP 地址具有两种形式：点分十进制和二进制形式。

点分十进制形式的 IP 地址是一个字符串，连同字符串结束符，字符串形式的 IP 地址的长度不超过 16 个字节。使用字符串形式 IP 地址的主要目的是为了方便用户，即：在程序输入输出 IP 地址时，使用点分十进制形式的 IP 地址。二进制形式的 IP 地址主要提供给套接字接口函数使用。因此，在网络通信程序中，常常需要在这种形式的 IP 地址之间进行转换。套接字编程接口主要提供了如下的接口函数来完成该项工作：

(1) unsigned int inet_addr (const char * cp);

函数的参数 cp 是一个指向存放有一个点分十进制表示的 IP 地址的字符串。返回值是与之对应的二进制形式的 IP 地址，且该地址已将 IP 地址转换为网络字节序。

当 cp 指向的字符串形式为“a.b.c.d”时，a、b、c、d 分别代表 IP 地址的 4 个字节；当字符串形式为“a.b.c”时，a、b 分别对应 IP 地址的高位两个字节，c 对应于后两个字节；当字符串形式为“a.b”时，a 被解释称 IP 地址的最高一个字节，而 b 则解释为后面的三个字节 24 位；当形式为“a”时，a 将直接被作为网络字节顺序的二进制表示的 IP 地址返回。

如果传入的字符串是一个非法的 IP 地址，函数返回值是常量 INADDR_NONE。

(2) int inet_aton(const char *cp, struct in_addr *in);

该函数把点分十进制字符串表示的 IP 地址转换为 32 位的无符号长整型数，并存放于参数 in 指向的 in_addr 结构变量中，该二进制形式的 IP 地址已转换为网络字节序。其中：

- cp：指向字符串形式的点分十进制形式的 IP 地址；
- in：指向 in_addr 结构变量的指针，该结构变量用来保存转换后的 IP 地址。

如果这个函数成功，函数的返回值非零。如果输入地址不正确则会返回零。

(3) char * inet_ntoa (struct in_addr in);

该函数将一个包含在 in_addr 结构变量 in 中的长整型 IP 地址转换点分十进制形式字符串。其中参数 in 是一个保存有 32 位二进制 IP 地址的 in_addr 结构变量，使用网络字节序。

函数调用成功返回一个字符指针，该指针指向一个 char 型缓冲区，缓冲区保存有由参数 in 的值转换而来的点分十进制表示的 IP 地址字符串。如果函数调用失败，则返回一个空指针 NULL。

2、实验内容及步骤

(1) 编写一个程序，该程序的功能有：

- 从键盘接收用户输入的点分十进制形式的 IP 地址；
- 把该 IP 地址转换为二进制形式，并保存到类型为 struct sockaddr_in 结构体变量 addr 中；

● 把保存在 addr 中的 IP 地址分别以二进制形式和点分十进制形式在屏幕上显示出来。

(2) 运行上述程序，把下面的表格填写完整。

输入	输出	
	二进制形式	点分十进制形式
10.132.219.5		
10.132.219		
10.132		
1234		

(3) 解释以上运行结果。

任务四：获取主机信息

1、原理

在编写网络通信程序时，有时需要获取计算机的名字、IP 地址等相关信息，为了实现这一目的，套接字编程接口提供了一系列函数来帮助应用程序完成相应的信息查询操作。以下介绍其中最常用的两个函数，一个用于获取主机名，另一个通过查询域名服务器来获取指定主机的 IP 地址、正式名称、别名等。

1、int gethostname(char *name, int namelen)

该函数用于查询本地计算机主机名字。其中：name 指向用于接收主机名字的缓冲区，也就是一个字符数组。namelen 指出缓冲区的大小，也就是字符数组 name 的大小。若缓冲区长度小于主机名长度，则会出错。使用该函数需要包含头文件 unistd.h。

2、struct hostent *gethostbyname(const char *name)

该函数通过查询域名服务器来获取指定名字的主机的信息。其中 name 指向主机的域名。调用成功，返回一个指向 hostent 结构的指针；调用失败，将返回一个空指针 NULL，使用 h_error 可以获得错误代码。使用该函数需要包含头文件 netdb.h。

返回指针指向的 hostent 结构由系统维护，该结构随线程的终结而消亡。应用程序不应该试图修改这个结构或者释放它的任何部分。hostent 结构定义如下：

```
struct hostent
{
   char *h_name;   /*主机的规范名*/
   char **h_aliases;  /*主机的别名*/
   int  h_addrtype;   /*主机ip地址的类型（AF_INET）*/
   int  h_length;     /*主机ip地址的长度（4）*/
   char **h_addr_list; /*主机ip地址列表（网络字节序）*/
};
#define h_addr h_addr_list[0]
```

以下程序清单 0-1 的程序演示了上述两个函数的用法。

程序清单 0-2 获取主机信息的程序示例

```
#include <stdio.h>
```

```
#include <netinet/in.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <unistd.h>
void main(void)
{
   char hostname[100];
   struct hostent *phost;
   char **pa;
   char **pip;
   int i;
   gethostname(hostname,100);
   printf("Local host name is: %s\n",hostname);
   printf("Please input host domain name:");
   scanf("%s",hostname);
   phost = gethostbyname(hostname);
   printf("The official name is:%s\n",phost->h_name);
   pa = phost->h_aliases;
   i = 0;
   while(*pa != NULL)
   {
         i++;
         printf("Aliases name%d is:%s\n",i,*pa);
         pa++;
   }
   printf("Address type:%d\n",phost->h_addrtype);
   printf("Address length:%d\n",phost->h_length);
   pip=phost->h_addr_list;
   i = 0;
   while(*pip != NULL)
   {
         i++;
         printf("IP address %d is:%s\n",i,inet_ntoa(*(struct in_addr*)(*pip)));
         pip++;
   }
}
```

在编写网络通信程序时，往往需要用户输入远程主机的地址。用户输入的地址既可能是远程主机的域名，也有可能是远程主机点分十进制形式的 IP 地址，这时为了确保能够获得远程主机的地址，可以采取如下图所示的处理流程。

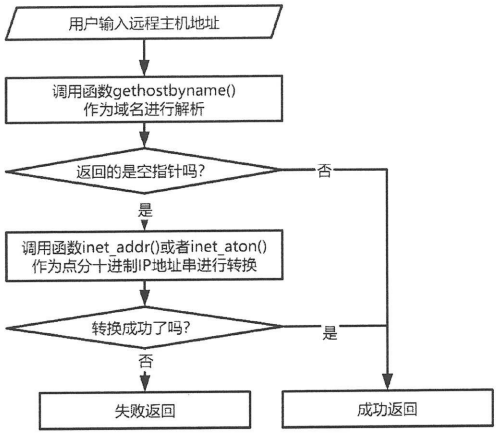


图 1 对用户输入地址的处理流程

2、实验内容及步骤

(1) 编写程序获取并输出本机的主机名。

(2) 编写一个能够获取主机信息的程序。该程序根据用户在程序运行时输入的主机域名，获取并输出该主机的正式名称、所有的主机别名的列表以及所有 IP 地址列表。其中 IP 地址要以点分十进制方式输出。

(3) 编写一个用于接收用户输入主机地址的程序，该程序既允许用户以域名方式输入主机地址，也允许用户以点分十进制形式来输入主机地址。